Problem-Framing Memo: Downtime Reduction Through Scheduler/API Integration

Domain B — Predictive Maintenance & Equipment Reliability | Problem 4: Downtime Reduction | Stage 1, Inuka Hackathon | 24 July 2026

The KPC Problem

KPC's pipeline maintenance workflow is largely manual: field issues get reported, but there is no automated, auditable path from “an asset needs attention” to “a ticket exists in the maintenance system with the right priority.” This creates two costs: **delay** between an issue being known and a technician being dispatched, and **invisibility** — no structured record connecting maintenance events to downtime hours, so the true cost of delay is never quantified. Problem 4 asks us to close that gap with automation, and prove the automation itself is reliable.

What the Data Already Tells Us

Our Stage 1 pipeline cleaned 600 work orders across 24 pump/valve assets in three depot zones (Mombasa, Nairobi, Kisumu), resolving three inconsistent date formats, 17 status-string variants, and a small number of physically impossible timestamps. Beyond cleaning, two automated checks already surface operational findings a downtime-reduction program should act on:

Finding	Result	Why it matters
Chronic-failure assets	6 of 24 assets (25%) at ≥ 1.2× fleet-average ticket volume	Priority list for condition-based maintenance rollout
Technician double-bookings	4 genuine scheduling conflicts detected	Direct cause of avoidable dispatch delay

Our Approach

1. ETL foundation (this stage): extract → clean → quality-gate → load into SQLite/MySQL, with structured logging and a 10-check automated quality gate enforced as a CI gate on every commit — currently passing 10/10.

2. Scheduler + API integration: a Cron-style scheduler reads the cleaned data, applies a priority rule set to open/overdue work orders, and creates tickets via a mock CMMS API (standing in for KPC's real ticketing system) — with retry logic and full performance monitoring (latency, success rate, retry counts) logged per run. Latest run: 264/264 tickets created, 181.7ms average latency.

3. Stage 2/3 direction: replace the mock API with real KPC access once granted; add a predictive layer forecasting which assets are *likely* to need attention soon, not just which already do — moving from reactive automation to genuinely proactive scheduling.

Why This Matters

If scheduler-driven ticket creation replaces manual reporting even partially, KPC gains an auditable, timestamped record of every maintenance trigger — the foundation needed to later quantify, and reduce, downtime hours with real evidence instead of estimates.